VOL. 07

Git

Repositories, branches, merges and the commands behind daily version control

Source	git-scm.com/book
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 07 - Git

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 6

Chapter map 6

Chapter 1 - Version control and the Git model 7

1.1 Summary 7

1.2 Concepts 7

What problem version control solves 7

Snapshots, not differences 8

The four areas of a project 8

Installing, configuring and getting help 8

1.3 Examples 8

1.4 Common mistakes and best practices 9

1.5 Practice exercises 9

Solutions 10

1.6 Mini test 10

Answer key and explanations 10

Chapter 2 - Repositories, staging and commits 11

2.1 Summary 11

2.2 Concepts 11

The daily loop 11

Reading `git status` 11

What `git add` really does 12

2.3 Examples 12

2.4 Common mistakes and best practices 14

2.5 Practice exercises 14

Solutions 14

2.6 Mini test 14

Answer key and explanations 15

Chapter 3 - Inspecting history, ignoring files and tagging 15

3.1 Summary 15

3.2 Concepts 16

The log is a graph, not a list 16

Comparing states and searching content 16

Ignore patterns and cleaning . . . 16
Tags as stable names . . . 17
3.3 Examples . . . 17
3.4 Common mistakes and best practices . . . 18
3.5 Practice exercises . . . 19
Solutions . . . 19
3.6 Mini test . . . 19
Answer key and explanations . . . 20
Chapter 4 - Undoing work, stashes and recovery . . . 20
4.1 Summary . . . 20
4.2 Concepts . . . 20
Three ways to undo . . . 20
The reflog . . . 21
Stash and worktrees . . . 21
4.3 Examples . . . 22
4.4 Common mistakes and best practices . . . 23
4.5 Practice exercises . . . 24
Solutions . . . 24
4.6 Mini test . . . 24
Answer key and explanations . . . 25
Chapter 5 - Branching and merging . . . 25
5.1 Summary . . . 25
5.2 Concepts . . . 25
What a branch is . . . 26
Fast-forward and merge commit . . . 26
The everyday loop . . . 26
5.3 Examples . . . 26
5.4 Common mistakes and best practices . . . 29
5.5 Practice exercises . . . 29
Solutions . . . 29
5.6 Mini test . . . 29
Answer key and explanations . . . 30
Chapter 6 - Conflicts and how to resolve them . . . 30
6.1 Summary . . . 30

6.2 Concepts . . . 31
When Git cannot decide . . . 31
The state of a conflicted merge . . . 31
Three ways out . . . 31
6.3 Examples . . . 31
6.4 Common mistakes and best practices . . . 34
6.5 Practice exercises . . . 34
Solutions . . . 34
6.6 Mini test . . . 35
Answer key and explanations . . . 35
Chapter 7 - Rebase, amend and rewriting history . . . 35
7.1 Summary . . . 35
7.2 Concepts . . . 36
Merge or rebase . . . 36
How a rebase works . . . 36
The interactive todo list . . . 36
7.3 Examples . . . 36
7.4 Common mistakes and best practices . . . 39
7.5 Practice exercises . . . 39
Solutions . . . 40
7.6 Mini test . . . 40
Answer key and explanations . . . 40
Chapter 8 - Remotes, hosting and team workflows . . . 41
8.1 Summary . . . 41
8.2 Concepts . . . 41
The vocabulary . . . 41
Fetch, pull, push . . . 41
Protocols, servers and hosting . . . 42
The GitHub flow . . . 42
8.3 Examples . . . 42
8.4 Common mistakes and best practices . . . 44
8.5 Practice exercises . . . 45
Solutions . . . 45
8.6 Mini test . . . 45
Answer key and explanations . . . 46

Chapter 9 - Inside Git: objects, references and configuration 46
9.1 Summary 46
9.2 Concepts 46
The four object types 46
References are the only names 47
Configuration, aliases and hooks 47
9.3 Examples 48
9.4 Common mistakes and best practices 50
9.5 Practice exercises 50
Solutions 50
9.6 Mini test 50
Answer key and explanations 51
Cheat sheet 51
10.1 Setup and configuration 51
10.2 Creating, recording and reading 51
10.3 Branches, merges, conflicts and tags 52
10.4 Rebase and history 52
10.5 Remotes, hosting and sharing 53
Final test 53
11.1 Final test answers 54
Sources 55
12.1 The Pro Git book (git-scm.com/book) 55
12.2 Manual pages used to check syntax and output 55
12.3 Related pages 55

How to use this volume

Prerequisites. A computer with a command line and a text editor. No previous version control experience is assumed. The commands are the same on Linux, macOS and Windows; where the output differs, the Windows result is shown.

Estimated study time. About 20 hours: two hours per chapter, plus the time you spend repeating the commands on your own files. Chapters 1 to 5 build the daily habit, chapters 6 and 7 are the branching skills every team expects, and chapters 8 and 9 explain how Git talks to other machines and where it stores data.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they really produce, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Every example was executed with Git 2.51 on Windows 11 in a scratch directory. Blocks tagged `bash` are commands you can type; blocks tagged `text` are the output those commands printed. Create your own playground with the three commands below and start again whenever you want.

shell

```bash
mkdir git-demo
cd git-demo
git init -b main
```

output

```text
Initialized empty Git repository in C:/Users/you/git-demo/.git/
```

Long absolute paths in the output are shortened, and repository URLs are shown as `/srv/git/project.git`; everything else is the exact output of the command.

About the source. The syllabus follows the official Pro Git book published at git-scm.com/book. Every example was written from scratch, run locally and checked against the manual pages. The book is licensed under a non-commercial share-alike licence, so it is used only as a list of topics.

Chapter map

#	Chapter	You will learn
1	Version control and the Git model	what version control solves, snapshots, the four areas, installing and configuring Git
2	Repositories, staging and commits	`git init`, `git add`, `git status`, `git diff`, `git commit` and the index
3	Inspecting history, ignoring files and tagging	log filters, `git diff`, `git blame`, `git grep`, `.gitignore`, `git clean`, tags

#	Chapter	You will learn
4	Undoing work, stashes and recovery	`git restore`, `git reset`, `git revert`, the reflog, `git stash`, worktrees
5	Branching and merging	branches, `git switch`, fast-forward merges, merge commits, `--no-ff`
6	Conflicts and how to resolve them	conflict markers, `--ours` and `--theirs`, resolving and committing a merge
7	Rebase, amend and rewriting history	`git rebase`, interactive rebase, squashing, cherry-pick, rebase versus merge
8	Remotes, hosting and team workflows	clone, fetch, pull, push, tracking, protocols, submodules, GitHub flow
9	Inside Git: objects, references and configuration	the object database, refs, plumbing, aliases, hooks, maintenance
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Version control and the Git model

1.1 Summary

- Version control records the history of a project so you can return to any earlier state and see who changed what.
- A centralised system shares one server, while a distributed system gives every developer a complete copy of the history.
- Git stores snapshots of the whole project, not lists of edits to individual files.
- A project has four working areas: the working tree, the index, the local repository and the remote repository.
- Git identifies every object by a checksum, so history cannot be changed silently.
- Installing Git also gives you a configuration system with three levels: system, global and repository.
- `git help` opens the manual for any command and is the fastest way to check a flag.

1.2 Concepts

What problem version control solves

A project is more than its current files. It is also the story of how those files came to be: which change fixed which bug, when a feature appeared, and what the code looked like before an experiment. Without a recorded history, questions like "what broke this week?" can only be

answered by guessing.

Git is distributed, which means a clone is a complete copy: committing, branching and inspecting history all work offline, and the network is only needed to exchange commits with other clones.

Snapshots, not differences

Many tools store the difference between two files. Git takes a snapshot of the project at each commit and stores an unchanged file as a reference to the earlier copy, so a clone of a long-lived project stays small and the same file in two branches costs no extra space.

The four areas of a project

output

```
 working tree        index            local repository      remote repository
+-----------+      +--------+        +---------------+     +----------------+
| files you | add  | staged | commit | commits,      |     | commits shared |
| edit now  | ---> | changes| ---->  | branches, tags| <-> | with a server  |
+-----------+      +--------+        +---------------+     +----------------+
      ^                                                    fetch / pull
      +---------------------------------------------------- push
```

Most beginner confusion comes from forgetting that a file can be in three states at once: modified in the working tree, staged in the index, and committed in the repository.

Term	Meaning
Commit	a snapshot plus metadata (author, date, message, parents)
Branch	a movable pointer to a commit
Tag	a fixed pointer to a commit, usually a release name
HEAD	the pointer that says which branch you are on
Origin	the default name for the remote you cloned from

Installing, configuring and getting help

Download the installer from git-scm.com/downloads and keep the defaults, then introduce yourself, because Git refuses to commit until both are known:

shell

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Three levels of configuration exist and the most specific one wins: `--system` applies to every user of the machine, `--global` to your account, and `--local` (the default inside a repository) to one project. `git help commit` opens the manual for a command, and `git add -h` prints a short option summary.

1.3 Examples

Example 1: check that Git is installed and configured.

shell

```
git --version
git config user.name
git config user.email
```

output

```
git version 2.51.2.windows.1
Ada Learner
ada@example.com
```

Example 2: create a repository and look at what the command created.

shell

```
mkdir git-demo
cd git-demo
git init -b main
ls -a
```

output

```
Initialized empty Git repository in C:/Users/you/git-demo/.git/
```

output

```
.  ..  .git
```

`-b main` names the first branch, and `.git` is the whole database: objects, references, hooks, configuration and the index. Deleting it deletes the history but leaves the working tree.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assuming `git commit` saves every file	only the staged snapshot is recorded	check `git status` before every commit
Committing without `user.name` and `user.email` set	the commit is refused	run the two `git config --global` lines once per machine
Running commands in the wrong directory	Git picks a different repository, or none	keep one project per directory and check the prompt
Deleting `.git` to "start over"	destroys every commit, branch and tag	create a new directory instead
Expecting the working tree to match the last commit	staged and unstaged changes are invisible there	read `git status` after every interruption
Writing a message such as `fix`	future readers cannot tell what changed	write what changed and why, in the imperative

1.5 Practice exercises

1. Name the four areas of a Git project and the command that moves content between each pair.
2. Which two configuration commands must every new developer run before the first commit?
3. What does `git init -b main` do that plain `git init` does not?
4. Where does Git keep the object database?

Solutions

1. Working tree, index, local repository, remote repository. `git add` moves working tree to index, `git commit` moves index to local repository, `git push` moves the local repository to the remote, and `git fetch` (or `git pull`) brings it back.
2. `git config --global user.name "..."` and `git config --global user.email "..."`.
3. It creates the repository and names the initial branch `main` in one step, instead of starting on the compiled-in default name.
4. Inside the `.git` directory.

1.6 Mini test

1. What is the main advantage of a distributed version control system? - A) a server decides who may commit - B) every clone holds the full history and works offline - C) files are stored as differences - D) history cannot be lost
2. What does Git record at each commit? - A) only the files that changed - B) the difference against the previous commit - C) a snapshot of the whole project - D) a backup of the working directory
3. Which area does `git add` write to? - A) the working tree - B) the index - C) the remote repository - D) the stash
4. True or false: because every object is identified by a checksum, a commit cannot be modified after it is created.
5. Which configuration level applies only to one project? - A) `--system` - B) `--global` - C) `--local` - D) `--user`
6. What is HEAD? - A) the most recent commit in the repository - B) the pointer to the branch you are currently on - C) the remote you cloned from - D) the first commit of the repository
7. Where does Git keep the object database? - A) in the working tree - B) in the index file only - C) inside the `.git` directory - D) on the remote server

Answer key and explanations

1. B - a clone contains the complete history, so commits, branches and logs work without a network.
2. C - Git stores snapshots; unchanged files are referenced again rather than rewritten.
3. B - `git add` copies the current content of a file into the index.
4. True - changing content changes its hash, so a correction is recorded as a new commit rather than an edit.

5. C - `--local` writes the repository's own `.git/config`.
6. B - HEAD points to the current branch, and the branch points to a commit.
7. C - objects, references, hooks and configuration all live under `.git`.

Chapter 2 - Repositories, staging and commits

2.1 Summary

- `git init` creates a repository; an existing project is joined with `git clone`.
- `git status` is the most useful command in Git: it lists untracked, modified and staged files.
- `git add` copies the current content of a file into the index, which is the snapshot the next commit will record.
- `git diff` compares the working tree with the index; `git diff --staged` compares the index with the last commit.
- `git commit -m "message"` stores the staged snapshot with your name, the date and a message.
- `git commit -a` stages every tracked modification, which is convenient but risky, and never includes new files.
- A commit message describes the change in the imperative mood, and a conventional prefix such as `feat:` or `fix:` makes the history readable.

2.2 Concepts

The daily loop

output

```
 edit files            stage the ones          record the snapshot
+-----------+  add    +-----------+  commit   +---------------+
| working   | ------> | index     | --------> | repository    |
| tree      |         |           |           | (history)     |
+-----------+         +-----------+           +---------------+
      |                                             |
      +------------- git restore / git reset ------+
```

The loop is intentionally small. Everything else in this volume is a way of looking at what the loop produced, undoing it, or moving it between repositories.

Reading `git status`

`git status` prints the branch, the relationship with its upstream, and three groups of files: untracked (seen on disk, never staged), modified (tracked and edited, but the edit is not in the index) and staged (the content of the last `git add`). `git status --short` prints the same information in two columns, which is why scripts prefer it.

output

```
 M hola.py
A  README.md
?? notes.txt
```

The first column is the index status, the second is the working tree status, and `??` means untracked.

What `git add` really does

`git add` does not record a change. It takes the bytes of the named files at that moment, writes them into the object database, and updates the index to point at them, which is why running it twice leaves one staged copy. `git restore --staged <path>` removes a path from the index again without touching the file on disk.

A good commit message has a short subject line of about 50 characters and, when needed, a blank line followed by a longer body. Conventional prefixes (`feat`, `fix`, `docs`, `chore`, `refactor`, `test`) are not a Git feature, but teams that agree on them can read a log like a changelog.

2.3 Examples

Example 1: create files and inspect the state.

shell

```
mkdir git-demo
cd git-demo
git init -b main
```

output

```
Initialized empty Git repository in C:/Users/you/git-demo/.git/
```

Write two files with your editor: `README.md` containing a short description, and `hola.py` containing one line of Python.

shell

```
git status --short
```

output

```
?? README.md
?? hola.py
```

Example 2: stage one file and commit it.

shell

```
git add README.md
git diff
git diff --staged
```

output

```
(nothing: git diff compares the working tree with the index)
```

output

```
diff --git a/README.md b/README.md
new file mode 100644
index 0000000..80e156c
--- /dev/null
+++ b/README.md
@@ -0,0 +1,3 @@
+# Project notes
+
+A tiny sample project used while learning Git.
```

shell

```
git commit -m "docs: add project readme"
```

output

```
[main (root-commit) fe7b859] docs: add project readme
 3 files changed, 4 insertions(+)
 create mode 100644 README.md
```

Add and commit the second file the same way: `git add hola.py` then `git commit -m "feat: add the hello script"`.

Example 3: edit a tracked file, compare the states, and commit.

Change `hola.py` so that it prints a second line, then compare the working tree with the index, unstage it, and stage it again.

shell

```
git status --short
git diff
git restore --staged hola.py
git status --short
git add hola.py
git commit -m "feat: print a second line"
```

output

```
 M hola.py
```

output

```
diff --git a/hola.py b/hola.py
index b376c99..75b970a 100644
--- a/hola.py
+++ b/hola.py
@@ -1 +1,3 @@
 print('hello')
+
+print('world')
```

output

```
(nothing printed)
```

output

```
 M hola.py
```

output

```
[main e15e740] feat: print a second line
 1 file changed, 2 insertions(+)
```

`git restore --staged` moved the file out of the index without losing the edit. To record every tracked modification in one step, `git commit -a -m "..."` works, but it never includes a file that was never added.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Committing with `git commit -a` before reading `git status`	unrelated edits slip into the snapshot	stage the files you mean to commit
Using `git add .` at the root of the home directory	stages thousands of files into the wrong repository	one repository per project directory
Writing `wip` or `fix` as a message	the log stops being readable	a subject line that says what changed, plus a body when needed
Running `git rm` and deleting the file by hand	Git records a staged deletion plus a working tree deletion	let `git rm` do both
Committing generated files such as logs or build output	history fills with noise	add a `.gitignore` rule, see chapter 3

2.5 Practice exercises

1. Write the command that shows, at once, what is staged, what is modified but not staged, and what is untracked.
2. What is the difference between `git diff` and `git diff --staged`?
3. A file is tracked and modified. Which two commands record the change in a new commit?
4. Why does `git commit -a` not include a newly created file?

Solutions

1. `git status` prints all three groups at once; `git status --short` prints the same in two columns; `git ls-files --others` lists only the untracked paths.
2. `git diff` compares the working tree with the index, `git diff --staged` compares the index with the last commit; comparing the working tree with the last commit uses `git diff HEAD`.
3. `git add <file>` then `git commit -m "..."`, or `git commit -a -m "..."` for the tracked modification in one step.
4. Because `-a` only stages files that Git already tracks; an untracked file has never been added and must be named with `git add`.

2.6 Mini test

1. Which command copies a file into the index? - A) `git commit` - B) `git add` - C) `git stage` - D) `git snapshot`
2. Which command shows the changes that the next commit would contain? - A) `git diff` - B) `git diff HEAD` - C) `git diff --staged` - D) `git log -p`
3. What does `??` mean in `git status --short` output? - A) the file is ignored - B) the file is staged for deletion - C) the file is untracked - D) the file has a merge conflict
4. True or false: `git commit -a` also stages files that have never been added.
5. What does `git restore --staged <file>` do? - A) deletes the file from the working tree - B) removes the file from the index but keeps it on disk - C) restores the file from the remote - D) reverts the file to the version in the previous commit
6. Which part of a commit message is conventionally kept under about 50 characters? - A) the body - B) the subject line - C) the trailer line - D) the message hash
7. Which command lists the files that the index tracks? - A) `git status` - B) `git ls-files` - C) `git list --index` - D) `git show --tracked`

Answer key and explanations

1. B - `git add` writes the current file content into the index.
2. C - `git diff --staged` compares the index with the last commit, which is exactly the pending change.
3. C - `??` is the short form of "untracked".
4. False - `-a` stages tracked modifications only; new files still need `git add`.
5. B - it only rewrites the index entry, so the file on disk is untouched.
6. B - the subject line stays short so that `git log --oneline` remains readable.
7. B - `git ls-files` prints the paths the index tracks.

Chapter 3 - Inspecting history, ignoring files and tagging

3.1 Summary

- `git log` walks the commits backwards from HEAD and prints the reachable history.
- `--oneline`, `--stat`, `-p` and `--format` change how much of each commit is printed.
- Filters narrow the log to a number of commits, an author, a path or a text pattern.
- `git show` prints one object, and `HEAD:<path>` reads a file as it was in a commit.
- `git blame` annotates a file with the commit that last touched each line, and `git grep` searches the tracked content.
- A `.gitignore` file lists patterns for files Git should leave alone, and it is itself tracked.
- Ignoring a file only keeps it out of the index; a file that is already tracked keeps being tracked.
- A tag names one commit forever, which makes tags the natural way to mark releases.

3.2 Concepts

The log is a graph, not a list

Each commit stores its parents. A normal commit has one parent, a merge has two or more, and the first commit has none, so `git log` prints a merge before the two lines it joined.

output

```
* 19cbd60 Merge branch topic-banner and keep the agreed banner
|\
| * 35a44dd feat: add banner from the topic
* | 5f932cb feat: add banner from main
|/
* 41bada3 chore: ignore build logs and temp files
```

`--graph` draws the lines, `--oneline` shortens the identifiers and messages, and `--all` includes every branch instead of only the current one.

Notation	Meaning
`HEAD~2`	two commits back along the first parent
`v1.1.0`	the commit a tag points at
`main..feature`	commits reachable from `feature` but not from `main`
`main...feature`	commits reachable from either but not from both

Comparing states and searching content

Command	Comparison
`git diff`	working tree against index
`git diff --staged`	index against HEAD
`git diff HEAD`	working tree against HEAD
`git log -p -2`	the patches of the last two commits

`git blame -L 5,5 <file>` prefixes one line with the commit that produced it, which tells you who to ask. `git grep -n "text"` searches the tracked content and prints file, line and match, while `git log -S"text" --oneline` finds the commits that added or removed the text instead.

Ignore patterns and cleaning

output

```
logs/
*.tmp
/build
!.env.example
```

`logs/` matches any directory named `logs` at any depth, `*.tmp` matches every file whose name ends in `.tmp`, `/build` matches only the `build` directory at the root of the project, and `!keep.tmp`

re-includes a file an earlier rule ignored. Rules are read top to bottom and the last matching rule wins. `git status --ignored` lists the excluded paths, which is the first place to look when a file you expected to be tracked is not. An ignore rule does not apply to a path that is already tracked: remove it from the index first with `git rm --cached <path>`. `git clean` deletes untracked files, so run the dry run first: `-n` lists what would go, `-d` adds directories, and `-X` restricts the run to ignored files.

Tags as stable names

A branch moves with every commit, so a branch name cannot identify a release. A tag does not move. A lightweight tag (`git tag v1.0.0`) is only a name pointing at a commit; an annotated tag (`git tag -a v1.1.0 -m "..."`) is a stored object with a tagger, a date and a message, which `git show` prints before the commit. `git describe --tags` turns the current commit into a name such as `v1.1.0-14-g51f2f7d`, which is exactly what a build number needs to say.

3.3 Examples

Example 1: log views, a patch, and a file inside a commit.

shell

```
git log --oneline
git log -p -1
git log --oneline -- README.md
```

output

```
6f73726 2026-10-01 feat: add the hello script
fe7b859 2026-10-01 docs: add project readme
```

output

```
commit e15e7407138912f07167ac3cc19b54954bb520ff
Author: Ada Learner <ada@example.com>
Date:   Thu Oct 1 09:00:00 2026 +0000

    feat: print a second line

diff --git a/hola.py b/hola.py
@@ -1 +1,3 @@
 print('hello')
+
+print('world')
```

output

```
fe7b859 docs: add project readme
```

Example 2: one path, one file inside a commit, and the author of a line.

shell

```
git log --oneline -- README.md
git show HEAD:hola.py
git blame -L 5,5 hola.py
git grep -n "world"
```

output

```
fe7b859 docs: add project readme
```

output

```
print('hello')

print('world')
```

output

```
0666e9b9 (Ada Learner 2026-10-01 09:00:00 +0000 5) print('world')
```

output

```
hola.py:5:print('world')
```

`HEAD:hola.py` names the version of the file recorded in that commit, not the version on disk, and a path after `--` restricts the log to commits that touched it.

Example 3: tags.

shell

```
git add .gitignore
git commit -m "chore: ignore build logs and temp files"
git tag v1.0.0
git tag -a v1.1.0 -m "First release with ignore rules"
git tag -n
git describe --tags
```

output

```
[main 41bada3] chore: ignore build logs and temp files
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```

output

```
(nothing printed)
```

output

```
v1.1.0          First release with ignore rules
```

output

```
v1.1.0-14-g51f2f7d
```

Publish the annotated tag with `git push origin v1.1.0`, and remember that `git push` alone does not send tags.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading a diff without knowing which two states it compares	you reason about the wrong change	state the pair: working tree, index or HEAD
Confusing `..` and `...` on branches	the two forms report different sets	two dots for "new commits", three dots for "since the fork"
Ignoring a file that is already tracked	the change has no effect	`git rm --cached <path>` and commit the removal
Running `git clean -fd` without `-n` first	untracked work is deleted	dry run, read the list, then clean
Marking releases with a branch	the name moves and no longer identifies the release	create an annotated tag per release
Expecting `git push` to send tags	releases never reach the remote	`git push origin <tag>` or `git push --tags`

3.5 Practice exercises

1. Write a command that prints the last three commits, one line each.
2. Which command shows the content of `README.md` as it was two commits ago?
3. What is the difference between `git log main..feature` and `git log main...feature`?
4. Write the `.gitignore` content that ignores every `.log` file, the `tmp` directory anywhere, and everything in `secret/` except `secret/README.md`.

Solutions

1. `git log --oneline -3`.
2. `git show HEAD~2:README.md`, or `git show HEAD~~:README.md`.
3. `main..feature` lists commits reachable from `feature` but not from `main`; `main...feature` lists commits reachable from either branch but not from both.
4. `*.log`, `tmp/`, `secret/*`, `!secret/README.md`.

3.6 Mini test

1. Which option prints one commit per line? - A) `-p` - B) `--stat` - C) `--oneline` - D) `--format=full`
2. What does `HEAD~2` refer to? - A) the second branch from HEAD - B) two commits back along the first parent - C) the commit two days ago - D) the second parent of HEAD
3. What does `git blame` add to each line of a file? - A) the author's email only - B) the commit, author, date and line number that last produced the line - C) the branch the line was written on - D) the line's checksum
4. Which notation means "commits in feature that are not in main"? - A) `main...feature` - B) `main..feature` - C) `feature^..main` - D) `main @feature`
5. What does `git show HEAD:notes.md` print? - A) the file on disk - B) the directory listing of the commit - C) the version of the file recorded in HEAD - D) the diff of the last commit

6. Why does adding a `.gitignore` rule not hide an already tracked file? - A) the file must be removed from the index first with `git rm --cached` - B) ignore rules only work on Windows - C) the file must be renamed - D) the rule must start with `!`
7. What is stored in an annotated tag but not in a lightweight tag? - A) the commit it points at - B) the tagger, the date and a message - C) the branch name - D) the author of the commit
8. True or false: `git tag v1.0` creates an annotated tag.

Answer key and explanations

1. C - `--oneline` abbreviates the identifier and prints only the subject.
2. B - `~` follows first parents, so `HEAD~2` is two commits back.
3. B - that is the information needed to ask the right person about a line.
4. B - two dots mean "reachable from the right side but not the left".
5. C - the `revision:path` syntax reads the file from a commit.
6. A - ignore rules are only consulted for paths that are not tracked.
7. B - the annotated tag is a stored object, which is why it can carry a message and a signature.
8. False - `git tag <name>` is lightweight; `-a` with `-m` creates the annotated form.

Chapter 4 - Undoing work, stashes and recovery

4.1 Summary

- `git restore` discards working tree changes, and `git restore --staged` moves a file out of the index.
- `git reset` moves the current branch to another commit and optionally updates the index and the working tree.
- `git reset --soft` keeps the index, `--mixed` (the default) clears it, and `--hard` also overwrites the working tree.
- `git revert` creates a new commit that undoes an earlier one, which is the safe undo for shared history.
- The reflog records where HEAD has been, so a commit that "disappeared" can usually be brought back.
- `git stash` shelves changes so you can switch context, and `git stash pop` brings them back.
- A linked worktree checks out a second branch into a second directory, sharing the same object database.

4.2 Concepts

Three ways to undo

The question is not only what you undo but who else already has the commit.

Goal	Command	History
Throw away an edit in the working tree	`git restore <file>`	untouched
Unstage a change	`git restore --staged <file>`	untouched
Move the branch pointer back	`git reset` with `--soft`, `--mixed` or `--hard`	rewritten
Undo the effect of a published commit	`git revert <commit>`	extended with a new commit

`git reset` changes where a branch points; the commits it used to reach stay in the database until garbage collection removes them, which is the reason recovery works.

output

```
 before            --soft            --mixed           --hard
+--------+        +--------+        +--------+        +--------+
| HEAD~1 |        | HEAD~1 |        | HEAD~1 |        | HEAD~1 |
| commit |        | staged |        | only   |        | clean  |
+--------+        +--------+        +--------+        +--------+
```

`git revert` does not delete a commit: it reads the patch the commit introduced, applies it backwards, and records the result as a new commit, so the message includes the identifier of the reverted commit and the audit trail stays intact.

The reflog

Every move of HEAD is appended to `.git/logs/HEAD`, with a reason and an index number, so reading it bottom-up tells the story of the session. To recover a commit, reset to its identifier or cherry-pick it onto the branch where it belongs.

output

```
7608f82 HEAD@{0}: reset: moving to HEAD~1
f5b8e96 HEAD@{1}: commit: chore: a commit to throw away
7608f82 HEAD@{2}: commit: chore: add the keep file
```

Warning. `git reset --hard` is the one command in this chapter that can destroy work that was never committed. Run `git status` first, and commit whenever you are unsure.

Stash and worktrees

`git stash` commits the working tree changes, resets the working tree to HEAD, and remembers the commit on a stack. Stashes never appear in a branch's history, so they suit work that is not ready to be a commit and nothing long-lived. `git stash push -m "msg"` labels an entry, `git stash list` shows the stack, and `git stash pop` restores the newest one. A linked worktree has its own working tree and index but shares the object database, so it is the tidy alternative to stashing when you must look at another branch.

4.3 Examples

Example 1: reset a small branch three ways.

shell

```
git switch -c reset-demo
echo v1 > demo.txt
git add demo.txt
git commit -m "demo: version one"
echo v2 > demo.txt
git add demo.txt
git commit -m "demo: version two"
git reset --soft HEAD~1
git status --short
```

output

```
[reset-demo eaed2ef] demo: version one
 1 file changed, 1 insertion(+)
 create mode 100644 demo.txt
```

output

```
M  demo.txt
```

`--soft` moved the branch but left the change staged, so it can be committed again with the same message.

shell

```
git reset --mixed HEAD~1
git status --short
git restore demo.txt
git status --short
```

output

```
 M demo.txt
```

output

```
?? scratch-notes.txt
```

`--mixed` left the same change as an unstaged edit, and `git restore` threw it away.

shell

```
echo v3 > demo.txt
git add demo.txt
git commit -m "demo: version three"
git reset --hard HEAD~1
git status --short
cat demo.txt
```

output

```
HEAD is now at eaed2ef demo: version one
```

output

```
v1
```

`--hard` printed the commit it landed on, the index is clean again, and the file holds the `v1` content from that commit.

Example 2: revert a commit and recover one that seems lost.

shell

```
git switch main
git revert --no-edit HEAD
git log --oneline -2
```

output

```
[main 5e30568] Revert "refactor: finish the notes file"
 1 file changed, 1 deletion(-)
```

output

```
5e30568 Revert "refactor: finish the notes file"
5603b30 refactor: finish the notes file
```

shell

```
echo "keep this one" > temp.txt
git add temp.txt
git commit -m "chore: add the keep file"
echo "keep this one, then break it" > temp.txt
git add temp.txt
git commit -m "chore: a commit to throw away"
git reset --hard HEAD~1
git reflog -3
git reset --hard HEAD@{1}
git log --oneline -2
```

output

```
HEAD is now at 7608f82 chore: add the keep file

7608f82 HEAD@{0}: reset: moving to HEAD~1
f5b8e96 HEAD@{1}: commit: chore: a commit to throw away
7608f82 HEAD@{2}: commit: chore: add the keep file
```

output

```
HEAD is now at f5b8e96 chore: a commit to throw away

f5b8e96 chore: a commit to throw away
7608f82 chore: add the keep file
```

The discarded commit was `HEAD@{1}`, so a reset to that entry brought it back.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`git reset --hard` on a branch someone else has	you rewrite published history and force a painful merge	use `git revert` for shared commits
Forgetting that `--hard` discards uncommitted edits	hours of work disappear	`git status` first, commit early
Amending a commit that was already pushed	the push is rejected and the history forked	amend only local commits
Using the stash as long-term storage	nobody else can see it and it is easy to forget	commit on a branch instead
Deleting a worktree directory by hand	the repository keeps a broken registration	`git worktree remove <path>`
Assuming a reset deletes commits	the objects survive, which is a source of confusion	verify with `git reflog`

4.5 Practice exercises

1. You staged a file by mistake. Which command removes it from the index and keeps it on disk?
2. What is the difference between `git reset --soft HEAD~1` and `git reset --hard HEAD~1`?
3. A commit was pushed yesterday and must be undone without rewriting history. Which command do you use?
4. You ran `git reset --hard HEAD~2` and lost a commit you needed. Where do you find it and how do you restore it?

Solutions

1. `git restore --staged <file>` (older documentation calls this `git reset HEAD <file>`).
2. Both move the branch one commit back. `--soft` keeps the index, so the change stays staged; `--hard` also resets the index and overwrites the working tree.
3. `git revert <commit>`, which records a new commit that undoes the change and can be pushed normally.
4. `git reflog` shows the previous positions of HEAD; reset or cherry-pick the commit identifier listed there, for example `git reset --hard HEAD@{1}`.

4.6 Mini test

1. Which command discards an unstaged edit to a tracked file? - A) `git restore <file>` - B) `git checkout <branch>` - C) `git reset --hard` - D) `git revert <file>`
2. What does `git reset --soft HEAD~1` leave in place? - A) nothing, the working tree is reset too - B) the index, so the change stays staged - C) the remote branch - D) a new commit
3. Which mode of `git reset` overwrites the working tree? - A) `--soft` - B) `--mixed` - C) `--hard` - D) `--keep`

4. True or false: `git revert` rewrites the commit it undoes.
5. Which command shows the recent positions of HEAD? - A) `git log --reflog` - B) `git reflog` - C) `git show HEAD` - D) `git status --history`
6. What is the risk of committing while HEAD is detached? - A) the commit is lost immediately - B) the commit belongs to no branch and can be stranded - C) the repository becomes corrupt - D) the working tree is reset
7. What does `git stash pop` do? - A) restores the newest stash entry and keeps it on the stack - B) restores the newest stash entry and removes it from the stack - C) lists the stack - D) deletes the newest entry without restoring it

Answer key and explanations

1. A - `git restore` copies the indexed version over the working tree file.
2. B - only the branch pointer moves, so the staged change is still there.
3. C - `--hard` resets the index and the working tree to the target commit.
4. False - revert records a new commit that reverses the change and leaves the original in place.
5. B - the reflog records every move of HEAD with a reason.
6. B - a commit created on a detached HEAD is only reachable through the reflog once you switch away.
7. B - `apply` restores and keeps; `pop` restores and drops; `drop` deletes.

Chapter 5 - Branching and merging

5.1 Summary

- A branch is a movable pointer to a commit, so creating one costs almost nothing.
- `git switch -c name` creates a branch and moves HEAD to it; `git switch name` moves between branches.
- Because HEAD is a pointer, switching branches replaces the working tree with the other line's content.
- A merge joins two lines of history: a fast-forward merge only moves the branch pointer, while a merge commit records two parents.
- `git merge --no-ff` keeps the merge commit even when a fast-forward would be possible, which preserves the shape of a feature.
- `git branch -d` deletes a merged branch; `-D` forces the deletion of an unmerged one.
- Branch names mirror the work: one branch per feature, bugfix or release, such as `feature/login`.
- A good workflow keeps `main` releasable and merges finished work into it deliberately.

5.2 Concepts

What a branch is

Git stores branches as small text files in `.git/refs/heads`, each containing one commit identifier. `git branch` lists them and the asterisk marks the one HEAD points at. Because the pointer is the whole branch, switching branches is a pointer change plus a working tree update, and two branches with identical content share every object.

Cheap branches change how you work: you open a branch for the task in front of you, commit as often as you like, and merge when it is done. Nothing on a branch affects `main` until you merge it.

Fast-forward and merge commit

output

```
    fast-forward                merge commit
  *--*--*--*               *--*--*        *--*--*
  main  |                     |              |
        *  (merge feature)     * feature  * feature
```

- If the target branch has no commits of its own since the branch was created, Git can simply move the pointer forward. No merge commit is needed.
- If both lines have commits, Git records a merge commit with two parents, which keeps the history of the feature visible.

`git merge --no-ff` forces the second shape, and `git merge --squash` collapses a whole branch into staged changes without a merge commit at all.

The everyday loop

shell

```
git switch main
git pull
git switch -c feature/name
```

Then work, add and commit; come back with `git switch main` and integrate with `git merge --no-ff feature/name` and `git push`. Review a branch before merging it with `git log main..feature` and `git diff --stat main...feature`, and delete the branch afterwards with `git branch -d`, which refuses a branch whose commits are not reachable from `main` unless you use `-D`.

5.3 Examples

Example 1: create a branch and fork the history.

shell

```
git branch feature-login
git switch feature-login
  # on feature-login: add a login helper to hola.py
git add hola.py
git commit -m "feat: add login helper"
git switch main
  # on main: add a Usage section to README.md
git add README.md
git commit -m "docs: describe usage"
git log --oneline --graph --all
```

output

```
Switched to branch 'feature-login'
```

output

```
[feature-login 4d76b4a] feat: add login helper
 1 file changed, 3 insertions(+)
```

output

```
[main bd21bda] docs: describe usage
 1 file changed, 4 insertions(+)
```

output

```
* bd21bda docs: describe usage
| * 4d76b4a feat: add login helper
|/
* 41bada3 chore: ignore build logs and temp files
* e15e740 feat: print a second line
* 6f73726 feat: add the hello script
* fe7b859 docs: add project readme
```

The graph shows the fork: the two lines share everything below it and each has one commit of its own.

Example 2: merge the feature branch.

shell

```
git merge feature-login
git log --oneline --graph
git branch -d feature-login
git branch -v
```

output

```
Merge made by the 'ort' strategy.
 hola.py | 3 +++
 1 file changed, 3 insertions(+)
```

output

```
*   e14e857 Merge branch 'feature-login'
|\
| * 4d76b4a feat: add login helper
* | bd21bda docs: describe usage
|/
* 41bada3 chore: ignore build logs and temp files
* e15e740 feat: print a second line
* 6f73726 feat: add the hello script
* fe7b859 docs: add project readme
```

output

```
Deleted branch feature-login (was 4d76b4a).
```

output

```
* main e14e857 Merge branch 'feature-login'
```

Both sides had new commits, so Git created a merge commit.

Example 3: force a merge commit with `--no-ff`.

shell

```
git switch -c topic-note
echo "an extra note" > EXTRA.md
git add EXTRA.md
git commit -m "docs: add an extra note"
git switch main
echo "The repository keeps the sample scripts at the top level." >> README.md
git add README.md
git commit -m "docs: describe the repository layout"
git merge --no-ff -m "Merge branch topic-note into the docs line" topic-note
git branch -d topic-note
```

output

```
Switched to a new branch 'topic-note'

[topic-note 23aba28] docs: add an extra note
 1 file changed, 1 insertion(+)
 create mode 100644 EXTRA.md
```

output

```
[main cbd2818] docs: describe the repository layout
 1 file changed, 4 insertions(+)
```

output

```
Merge made by the 'ort' strategy.
 EXTRA.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 EXTRA.md
```

output

```
Deleted branch topic-note (was 23aba28).
```

The merge commit keeps the topic visible as one unit even when a fast-forward would have been possible.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Working directly on `main` for days	no way to abandon a half-finished idea	one branch per task, merged when done
Leaving ten stale local branches	nobody can tell what is in progress	delete a branch as soon as it is merged
Using `git branch -D` to clean up	commits that were never merged are lost	merge or tag first, then `-d`
Merging while the working tree is dirty	unrelated edits join the merge commit	commit or stash before switching
Naming every branch `test` or `tmp`	the log becomes unreadable	use `type/short-description` names
Merging a feature without looking at its commits	a surprise lands on `main`	review with `git log main..feature` first

5.5 Practice exercises

1. Which command creates a branch called `fix/login` and moves to it?
2. Why does `git switch` sometimes refuse to switch branches?
3. What is the difference between a fast-forward merge and a merge commit, and which option forces the merge commit?
4. Write the command that lists the commits a branch adds compared with `main`.

Solutions

1. `git switch -c fix/login`, or `git branch fix/login` followed by `git switch fix/login`.
2. Because the switch would overwrite local changes: the same file is modified in both branches. Commit, stash, or discard the changes first, which is what `git status` explains.
3. A fast-forward merge moves the branch pointer because the target had no new commits; a merge commit records two parents because both lines moved. `git merge --no-ff` forces the merge commit.
4. `git log --oneline main..fix/login` for the list and `git diff --stat main...fix/login` for a summary.

5.6 Mini test

1. Which command creates a branch and moves to it? - A) `git branch name` - B) `git switch -c name` - C) `git switch name` - D) `git checkout main`

2. What is a branch? - A) a copy of the files - B) a movable pointer to a commit - C) a snapshot stored separately - D) a tag with a different name
3. When does Git perform a fast-forward merge? - A) when both branches have new commits - B) when the target branch has no commits since the branch was created - C) when there is a conflict - D) when `--no-ff` is given
4. Which option forces a merge commit? - A) `--squash` - B) `--ff-only` - C) `--no-ff` - D) `--abort`
5. True or false: a merge commit has two parents.
6. What does `git branch -v` add to the branch list? - A) the last commit of each branch - B) the remote tracking branch - C) the date of the repository - D) nothing, it is the same as `git branch`
7. Why does Git refuse `git branch -d` on an unmerged branch? - A) the branch is checked out - B) the commits are not reachable from the current branch - C) the branch has a tag - D) the name is invalid
8. Which command draws the project's shape including every branch? - A) `git log --oneline` - B) `git log --graph --oneline --all` - C) `git branch --all --graph` - D) `git show --graph`

Answer key and explanations

1. B - `switch -c` creates and checks out in one step.
2. B - the branch is a small reference file holding one commit identifier.
3. B - moving the pointer forward is enough, so no merge commit is created.
4. C - `--no-ff` records a merge commit even when a fast-forward is possible.
5. True - the merge commit keeps both lines of the history reachable.
6. A - `-v` prints the short identifier and the subject of each branch's tip.
7. B - the deletion would make commits unreachable, so Git asks for `-D`.
8. B - `--graph` draws the lines and `--all` includes every branch.

Chapter 6 - Conflicts and how to resolve them

6.1 Summary

- A conflict happens when the same lines of a file were changed differently on two lines of history.
- Git stops the merge, marks the file as unmerged, and writes conflict markers into it.
- `git status` lists the unmerged paths, and `git diff` shows a combined diff of both sides.
- `git checkout --ours <file>` or `--theirs <file>` takes one side wholesale, which is right for generated files and wrong for hand-edited ones.
- The resolution is an edit: keep the wanted lines, delete the markers, then `git add` and `git commit`.
- `git merge --abort` returns the repository to the state before the merge started.

- A conflict is information about a disagreement, so ask the person on the other side when the answer is not obvious.

6.2 Concepts

When Git cannot decide

A merge combines changes. If one branch changed a file and the other did not, the changed version wins. The problem starts when both branches changed the same region of the same file: Git has two valid versions and no rule to choose, so it asks a human.

output

```
<<<<<<< HEAD
banner from main
=======
banner from the topic branch
>>>>>>> topic-banner
```

Everything above `=======` comes from the current branch, here `HEAD`; everything below it comes from the branch being merged, which the last line names. Both markers and the separator must be deleted by hand, or with a merge tool.

The state of a conflicted merge

Where to look	What it tells you
`git status`	the unmerged paths, grouped by the kind of conflict
`git diff`	a combined diff showing both sides at once
`git log --merge --oneline`	the commits from both sides that touched the file
`git checkout --ours` or `--theirs`	replace the file with one side

Conflicts come in a few kinds: `both modified` means the same lines changed on both sides, `added by both` (short status `AA`) means a new file with the same name appeared twice, and `deleted by them` means one side deleted a file the other edited.

Three ways out

output

```
    resolve the file          take one side          give up
    edit, add, commit    |    checkout --ours/theirs |  merge --abort
                         |    add, commit            |
```

Taking one side wholesale is legitimate when a file is generated or when the other side's version is known to be complete, but it silently drops the other work, so it is the last choice for hand-written code.

6.3 Examples

Example 1: create the disagreement.

shell

```
git switch -c topic-banner
echo "banner from the topic branch" > banner.txt
git add banner.txt
git commit -m "feat: add banner from the topic"
git switch main
echo "banner from main" > banner.txt
git add banner.txt
git commit -m "feat: add banner from main"
git merge topic-banner
```

output

```
Switched to a new branch 'topic-banner'
```

output

```
[topic-banner 35a44dd] feat: add banner from the topic
 1 file changed, 1 insertion(+)
 create mode 100644 banner.txt
```

output

```
[main 5f932cb] feat: add banner from main
 1 file changed, 1 insertion(+)
 create mode 100644 banner.txt
```

output

```
Auto-merging banner.txt
CONFLICT (add/add): Merge conflict in banner.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Because the file did not exist in the common ancestor, the conflict is `add/add` rather than `content`.

Example 2: inspect the state.

shell

```
git status
cat banner.txt
git diff
```

output

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
both added:      banner.txt

no changes added to commit (use "git add" and/or "git commit -a")

$ cat banner.txt
<<<<<<< HEAD
banner from main
=======
banner from the topic branch
>>>>>>> topic-banner
```

output

```
diff --cc banner.txt
index 90a0256,c8ff651..0000000
--- a/banner.txt
+++ b/banner.txt
@@@ -1,1 -1,1 +1,5 @@@
++<<<<<<< HEAD
 +banner from main
++=======
+ banner from the topic branch
++>>>>>>> topic-banner
```

Example 3: resolve it, or take one side.

shell

```
echo "banner text agreed by both branches" > banner.txt
git add banner.txt
git commit -m "Merge branch topic-banner and keep the agreed banner"
git branch -d topic-banner
```

output

```
[main d696584] Merge branch topic-banner and keep the agreed banner
```

output

```
Deleted branch topic-banner (was 35a44dd).
```

The message was written by hand instead of taken from the default template, which is the moment to say what was decided and why. If instead the answer is simply "take the incoming version", `git checkout --theirs banner.txt` writes that file, but nothing is resolved until `git add` runs, and `git status --short` keeps showing `AA banner.txt`.

Example 4: give up safely.

shell

```
git merge topic-banner
git merge --abort
git status --short
```

output

```
error: Merging is not possible because you have unmerged files.
fatal: Exiting because of an unresolved conflict.
```

output

```
(nothing printed: the working tree and the index are restored)
```

output

```
?? scratch-notes.txt
```

Git refuses to start a second merge while one is unresolved. `--abort` returns the working tree and the index to the state before the merge, and never touches the commits you made earlier.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Committing a file that still contains `<<<<<<<`	the markers reach production and break the build	search for the markers before `git add`
Choosing `--theirs` without reading it	the other side's work is silently dropped	read both sides, or ask the other author
Fixing a conflict by re-typing whole files	small edits from the other branch are lost	keep the other side's lines unless you have a reason not to
Running `git add .` to finish a merge	unrelated files join the merge commit	add only the conflicted paths
Rebasing a branch that others share	the same conflicts appear for everyone	agree on merge or rebase as a team before publishing

6.5 Practice exercises

1. Which two areas of a conflicted file are separated by `=======`, and what does the last marker line name?
2. Which command lists the files that are still unmerged?
3. Write the commands that resolve a conflict in `config.yml` by editing, then record the merge.
4. How do you cancel a merge that has already produced conflicts?

Solutions

1. Above the separator is the current branch (`HEAD`), below it is the branch being merged; the last line names that branch.
2. `git status`, which groups the paths under "Unmerged paths", or `git diff --name-only --diff-filter=U`.

3. Edit the file, remove the markers, `git add config.yml`, then `git commit` (without `-m` the prepared merge message is reused).
4. `git merge --abort`.

6.6 Mini test

1. What do the lines between `<<<<<<< HEAD` and `=======` contain? - A) the common ancestor version - B) the version on the current branch - C) the version from the branch being merged - D) the merged result
2. Which command marks a conflicted file as resolved? - A) `git commit <file>` - B) `git add <file>` - C) `git resolve <file>` - D) `git checkout <file>`
3. Which short status means a file was added on both sides of a merge? - A) `MM` - B) `AA` - C) `??` - D) `DD`
4. True or false: `git merge --abort` deletes the commits made before the merge.
5. What is an `add/add` conflict? - A) the same file was edited on both sides - B) a file with the same name was created on both sides - C) a file was deleted on both sides - D) a file was renamed twice
6. Which command takes the incoming branch's version of a file without editing it? - A) `git checkout --ours -- <file>` - B) `git checkout --theirs -- <file>` - C) `git restore --merged --<file>` - D) `git take-theirs <file>`
7. Which command shows the commits from both sides that touched a conflicted file? - A) `git log --merge -- <file>` - B) `git blame --merge` - C) `git diff --both` - D) `git show --conflict`

Answer key and explanations

1. B - the upper section is the current branch, `HEAD`.
2. B - staging a file records the resolution in the index and clears the unmerged state.
3. B - `AA` means the path was added on both sides; `UU` would mean both sides modified the same lines.
4. False - it restores the working tree and index to the state before the merge; earlier commits are untouched.
5. B - the file did not exist in the common ancestor and appeared on both sides.
6. B - `--theirs` loads the version from the branch being merged; `--ours` keeps the current branch's version.
7. A - `--merge` limits the log to the commits involved in the current merge.

Chapter 7 - Rebase, amend and rewriting history

7.1 Summary

- Rebasing replays the commits of one branch on top of another, producing a straight line of history.
- A merge keeps the fork and records a merge commit; a rebase hides the fork and creates new commit identifiers.
- The interactive rebase opens a todo list where commits can be reordered, reworded, squashed, split or dropped.
- Squashing turns several small commits into one clean commit before the work is shared.
- `git commit --amend` is the smallest rewrite: it replaces the last commit.
- `git cherry-pick` copies one commit from another branch onto the current one.
- Rewriting is safe only for history that has not been pushed; after a push, use `git revert` instead.
- A rebase conflict is resolved exactly like a merge conflict, then continued with `git rebase --continue`.

7.2 Concepts

Merge or rebase

Merge	Rebase
keeps every commit exactly as written	rewrites the merged commits with new identifiers
records the fork in the graph	produces a straight line
conflicts are solved once	conflicts may be solved commit by commit
right for shared branches	right for a private branch before it is shared

A common team rule: rebase your own feature branch onto the latest `main` to keep it up to date, then merge it with `--no-ff` so the merge commit documents the feature.

How a rebase works

The interactive todo list

`git rebase -i HEAD~3` opens a list of the last three commits.

output

```
pick 95cbfd8 wip: add the scratch file
pick b981897 wip: finish the scratch file

  # Rebase 5603b30..b981897 onto 5603b30 (2 commands)
```

`pick` keeps a commit, `reword` keeps it and edits the message, `edit` stops after it, `squash` merges it into the previous commit and keeps both messages, `fixup` merges it and keeps only the previous message, and `drop` removes it. Changing `pick` to `squash` on the second line, saving and closing the editor is the classic way to turn two `wip` commits into one.

7.3 Examples

Example 1: fork the history, then replay one line onto the other.

shell

```shell
git switch -c refactor
echo one > notes.md
git add notes.md
git commit -m "refactor: start the notes file"
printf 'one\ntwo\n' > notes.md
git add notes.md
git commit -m "refactor: finish the notes file"
git switch main
echo "Run python hola.py, then read the notes." >> README.md
git add README.md
git commit -m "docs: mention the notes file"
git switch refactor
git rebase main
git log --oneline --graph --all
git switch main
git merge --ff-only refactor
git branch -d refactor
```

output

```
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/refactor.
```

output

```
* 5603b30 refactor: finish the notes file
* 1796d68 refactor: start the notes file
* d5365dc docs: mention the notes file
* 41bada3 chore: ignore build logs and temp files
* e15e740 feat: print a second line
* 6f73726 feat: add the hello script
* fe7b859 docs: add project readme
```

output

```
Updating d5365dc..5603b30
Fast-forward
 notes.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 notes.md
```

The fork is gone: the two `refactor` commits now sit directly on top of the note about the docs, and `main` could simply be moved forward.

Example 2: squash two `wip` commits into one.

shell

```
git switch -c tidy-up
echo alpha > scratch.txt
git add scratch.txt
git commit -m "wip: add the scratch file"
printf 'alpha\nbeta\n' > scratch.txt
git add scratch.txt
git commit -m "wip: finish the scratch file"
git log --oneline -3
GIT_SEQUENCE_EDITOR="sed -i '2s/^pick/squash/'" git rebase -i HEAD~2
git log --oneline -3
git switch main
git branch -D tidy-up
```

output

```
b981897 wip: finish the scratch file
95cbfd8 wip: add the scratch file
5603b30 refactor: finish the notes file
```

output

```
Rebasing (2/2)
[detached HEAD 604005d] wip: add the scratch file
 Date: Fri Oct 9 09:00:00 2026 +0000
 1 file changed, 2 insertions(+)
 create mode 100644 scratch.txt
Successfully rebased and updated refs/heads/tidy-up.
```

output

```
604005d wip: add the scratch file
5603b30 refactor: finish the notes file
1796d68 refactor: start the notes file
```

One commit remains, with the content of both, and the identifier changed because its parent did. The editor instruction above drives the todo list without opening an editor; in normal use you edit the list by hand.

Example 3: amend the last commit, and copy a commit to another branch.

shell

```
git switch main
echo "Contributors: Ada Learner" > CONTRIBUTING.md
git add CONTRIBUTING.md
git commit -m "docs: add the contributor list"
echo "v1.1.0 first sample release" > CHANGELOG.md
git add CHANGELOG.md
git commit --amend --no-edit
git switch -c hotfix
echo "Run python hola.py, read the notes, then run the tests." > README.md
git add README.md
git commit -m "fix: reword the usage line"
git switch main
git cherry-pick hotfix
git branch -D hotfix
```

output

```
[main 08300b8] docs: add the contributor list
 1 file changed, 1 insertion(+)
 create mode 100644 CONTRIBUTING.md
```

output

```
[main 8e1fe98] docs: add the contributor list
 Date: Mon Oct 19 09:00:00 2026 +0000
 2 files changed, 2 insertions(+)
 create mode 100644 CHANGELOG.md
 create mode 100644 CONTRIBUTING.md
```

output

```
[hotfix 51f2f7d] fix: reword the usage line
 1 file changed, 1 insertion(+), 1 deletion(-)
```

output

```
[main 51f2f7d] fix: reword the usage line
 Date: Sat Oct 17 09:00:00 2026 +0000
 1 file changed, 1 insertion(+), 1 deletion(-)
```

The amended commit has a new identifier, so the original is no longer reachable from `main` - which is why amending after a push causes trouble. The cherry-pick put the same change on `main` under a new identifier, and the `hotfix` branch is no longer needed.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Rebasing a branch that a colleague is using	their history no longer matches yours	rebase only your own unpublished branch
Force pushing a shared branch	everyone else must reset their work	communicate, and use `--force-with-lease`
Rebasing to tidy commits that are already reviewed	the review is attached to commits that no longer exist	squash before asking for a review
Solving a conflict during a rebase by choosing one side blindly	the same conflict returns on the next commit	read both sides, as in a merge
Using `fixup` without checking the result	commits land on the wrong parent	inspect the todo list before saving
Amending a commit that was pushed	the push is rejected and the history forks	`git revert` instead

7.5 Practice exercises

1. What is the difference between `git merge feature` and `git rebase feature`?
2. Which interactive rebase command combines a commit into the previous one and discards its message?

3. Why does the identifier of a commit change after a rebase?
4. You rewrote a branch that you had already pushed. What must you do, and what must your collaborators do?

Solutions

1. `git merge feature` integrates the branch into the current one and records a merge commit; `git rebase feature` replays the current branch's commits on top of `feature` and moves the branch, producing a straight line.
2. `fixup`, as opposed to `squash`, which combines the messages.
3. Because a commit records its parents: a new parent means new content to hash, so every replayed commit gets a new identifier.
4. Push with `git push --force-with-lease` and tell the team. Collaborators re-create their branch from the rewritten remote, or rebase their own work onto it.

7.6 Mini test

1. What does a rebase do to the commits it replays? - A) leaves them untouched - B) rewrites them with new identifiers - C) deletes them - D) converts them into tags
2. Which command replays the current branch onto `main`? - A) `git merge main` - B) `git rebase main` - C) `git cherry-pick main` - D) `git reset main`
3. Which interactive command keeps a commit but lets you edit its message? - A) `edit` - B) `reword` - C) `squash` - D) `fixup`
4. What does `git commit --amend --no-edit` do? - A) creates an empty commit - B) replaces the last commit, adding the staged content and keeping the message - C) undoes the last commit - D) pushes the last commit
5. True or false: rebasing a branch that others have cloned is safe as long as you tell them afterwards.
6. Which command copies one commit from another branch? - A) `git merge --squash` - B) `git cherry-pick` - C) `git rebase -i` - D) `git apply`
7. How do you resume a rebase that stopped on a conflict? - A) `git rebase --restart` - B) resolve the file, `git add` it, then `git rebase --continue` - C) `git commit` then `git rebase --done` - D) `git merge --continue`
8. Which option turns a long-lived branch into one commit when merging? - A) `--no-ff` - B) `--squash` - C) `--ff-only` - D) `--abort`

Answer key and explanations

1. B - the replayed commits get new parents, therefore new identifiers.
2. B - the current branch's commits are replayed on top of `main`.
3. B - `reword` reopens the message editor for that commit.
4. B - amend replaces the last commit with the index plus the staged changes.
5. False - rewriting published history breaks every clone until it is reset; communicate first and prefer `revert` after a push.

6. B - cherry-pick applies the change of one commit to the current branch.
7. B - the conflict is resolved like any other, then the rebase continues with the remaining commits.
8. B - `--squash` stages the whole branch's changes as one commit.

Chapter 8 - Remotes, hosting and team workflows

8.1 Summary

- A remote is another clone of the same project, referenced by a name such as `origin`.
- `git remote -v` lists the fetch and push URLs of every remote; `git remote show origin` explains the tracking setup.
- `git clone` copies a remote into a new directory, creates `origin`, and sets up the local branch to track its counterpart.
- `git fetch` downloads objects and updates the remote-tracking branches without touching your work.
- `git pull` is a fetch followed by a merge, or a rebase if configured that way.
- `git push` publishes your local commits, and `-u` records the upstream branch for the first time.
- Git speaks several protocols; SSH and HTTPS are the ones a beginner will use, and a host uses a bare repository.
- The GitHub flow is: branch, commit, push, open a pull request, review, merge.

8.2 Concepts

The vocabulary

output

```
 your clone                        the shared remote
+------------------+               +------------------+
| main             | <--- push --- | refs/heads/main  |
| origin/main      | --- fetch --> | refs/heads/feature|
| origin/feature   |               | refs/tags/v1.1.0 |
+------------------+               +------------------+
```

A remote is a named URL. A remote-tracking branch is a local copy of a branch that lives on the remote, written as `origin/main` and updated only by `fetch`. The upstream is the branch your branch compares itself with, stored in `branch.<name>.remote` and `branch.<name>.merge`.

Fetch, pull, push

- `git fetch origin` downloads everything new and updates `origin/*`. Your branches do not move.
- `git pull origin main` fetches and then merges `origin/main` into the current branch.

- `git push origin main` sends your `main` commits to the remote's `main`.

Fetching is always safe, because it only adds information. Pulling changes your working tree, and pushing changes the shared history, so a push can be rejected when someone else pushed first.

output

```
    git status -sb
    ## main...origin/main [ahead 1]
```

`[ahead 1]` means one local commit that the remote does not have; `[behind 1]` means a pull would move your branch without any local work.

Protocols, servers and hosting

Protocol	Typical URL	Notes
Local path	`/srv/git/project.git`	fast, for a disk you control
SSH	`git@example.com:team/project.git`	key based, no password prompts after setup
HTTPS	`https://example.com/team/project.git`	works through proxies, needs a credential helper

A bare repository has no working tree, which is why servers use them: it holds the history and accepts pushes, and `git init --bare -b main /srv/git/project.git` starts one. Hosted services such as GitHub, GitLab and Bitbucket add pull requests, issues, access control and CI on top of the same commands, and a credential helper remembers HTTPS passwords so you do not type them for every push. A submodule records another repository at a fixed commit as a gitlink of mode `160000`, so a fresh clone needs `git submodule update --init --recursive` to populate the directory, and a bundle is a file that contains a whole history and can be cloned from.

The GitHub flow

Fork or clone, create a branch for one change, commit in small steps, push the branch, open a pull request, address the review, merge, and delete the branch. The pull request is a conversation, not a command, and `git fetch --prune` keeps the local list of remote branches honest.

8.3 Examples

Example 1: clone the project, and push a new branch from the clone.

shell

```
git clone /srv/git/project.git work-two
cd work-two
git switch -c feature-remote
echo "a change made in a second clone" > changes.txt
git add changes.txt
git commit -m "feat: change from a second clone"
git push -u origin feature-remote
```

output

```
Cloning into 'work-two'...
done.
```

output

```
Switched to a new branch 'feature-remote'
```

output

```
[feature-remote 0faf21f] feat: change from a second clone
 1 file changed, 1 insertion(+)
 create mode 100644 changes.txt
```

output

```
branch 'feature-remote' set up to track 'origin/feature-remote'.
To C:\Users\you\srv\git\project.git
 * [new branch]      feature-remote -> feature-remote
```

A clone copies the whole history, creates `origin`, checks out the default branch and writes the tracking configuration.

Example 2: pull the colleague's branch into the first clone.

shell

```
  # back in the first clone
git fetch origin
git branch -r
git pull --ff-only origin feature-remote
git log --oneline --decorate --graph -3
```

output

```
From /srv/git/project
 * [new branch]      feature-remote -> origin/feature-remote

  origin/HEAD -> origin/main
  origin/feature-remote
  origin/main
```

output

```
From /srv/git/project
 * branch            feature-remote -> FETCH_HEAD
Updating f5b8e96..0faf21f
Fast-forward
 changes.txt | 1 +
 1 file changed, 1 insertion(+)
```

output

```
* 0faf21f (HEAD -> main, origin/feature-remote) feat: change from a second clone
* f5b8e96 (origin/main, origin/HEAD) chore: a commit to throw away
* 7608f82 chore: add the keep file
```

`--ff-only` refuses the pull if a fast-forward is impossible, which is the safe default: you then look before merging or rebasing by hand.

Example 3: a submodule and a bundle.

shell

```shell
git submodule add /srv/git/library.git vendored/lib
git status --short
git submodule status
git commit -m "chore: add the vendored library"
git bundle create project.bundle --all
git bundle verify project.bundle
```

output

```
Cloning into 'C:/Users/you/git-demo/vendored/lib'...
done.
```

output

```
A  .gitmodules
A  vendored/lib
?? scratch-notes.txt
```

output

```
 f5b8e9639559fc62efb873d3561e09a95798e816 vendored/lib (v1.1.0-12-gf5b8e96)
```

output

```
[main 76d7699] chore: add the vendored library
 2 files changed, 4 insertions(+)
 create mode 100644 .gitmodules
 create mode 160000 vendored/lib
```

Mode `160000` is the gitlink: the parent records the submodule's commit, not its files.

output

```
(nothing printed while the bundle is written)
```

output

```
The bundle contains these refs:
76d7699ea5278dba05b0334bf241ae08cd784ad9 refs/heads/main
f5b8e9639559fc62efb873d3561e09a95798e816 refs/heads/feature-remote
41bada3eb5282d327bac47461941a02054ac7b03 refs/tags/v1.1.0
The bundle records a complete history.
C:/Users/you/git-demo/project.bundle is okay
```

A bundle can be carried on a USB stick and cloned like any other repository.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Pulling before pushing on a shared branch	the push is rejected as non-fast-forward	`git fetch`, compare, then pull or rebase
Working for a week on a branch	conflicts grow and review is hard	commit and push daily, rebase often

Mistake	Why it hurts	Best practice
Pushing to `main` directly	no review, no CI, no discussion	push a branch and open a pull request
Forgetting that `pull` merges by default	surprise merge commits	set `pull.rebase true` or use `--ff-only`
Keeping stale remote-tracking branches	the branch list lies	`git fetch --prune` regularly
Cloning a project with submodules and getting empty directories	the build fails	`git submodule update --init --recursive`
Force pushing a shared branch	other clones diverge and work is duplicated	coordinate first, prefer `--force-with-lease`

8.5 Practice exercises

1. What is the difference between `git fetch` and `git pull`?
2. What does `git push -u origin feature` do that `git push origin feature` does not?
3. A colleague deleted a branch on the remote after merging it. Which command cleans it from your clone?
4. Why does a server usually host a bare repository?

Solutions

1. `fetch` downloads objects and updates `origin/*` only; `pull` also integrates the fetched branch into the current branch with a merge or a rebase.
2. It records the upstream, so `git status`, `git pull` and `git push` work without naming the branch from then on.
3. `git fetch --prune`, which removes the remote-tracking branch.
4. Because a server has no files to edit, so a working tree would only be a liability; the bare repository holds the history and accepts pushes.

8.6 Mini test

1. Which command downloads changes without modifying your branch? - A) `git pull` - B) `git fetch` - C) `git clone` - D) `git push`
2. What does `origin` refer to? - A) the first commit - B) the default name of the remote you cloned from - C) the local branch - D) the tag list
3. Which command publishes a local branch for the first time and records its upstream? - A) `git push --new origin branch` - B) `git push -u origin branch` - C) `git remote add branch` - D) `git clone -u branch`
4. True or false: `git fetch` can fail if the remote history diverged from yours.
5. What does `## main...origin/main [ahead 1]` mean? - A) the remote is one commit ahead of you - B) you have one commit that the remote does not have - C) the branches have

diverged - D) the branch was deleted on the remote

6. Which option makes `git pull` refuse anything but a fast-forward? - A) `--no-ff` - B) `--ff-only` - C) `--squash` - D) `--prune`
7. What is a bare repository? - A) an empty repository - B) a repository with no working tree - C) a repository without history - D) a repository that only accepts pulls
8. Which mode marks a submodule in `git ls-files -s`? - A) `100644` - B) `100755` - C) `160000` - D) `040000`

Answer key and explanations

1. B - fetch only adds objects and updates the remote-tracking branches.
2. B - `origin` is a conventional name, not a special value.
3. B - `-u` is short for `--set-upstream`.
4. False - fetching only adds information and cannot conflict; a pull or a push is where the divergence matters.
5. B - `ahead` counts your local commits that the remote lacks.
6. B - `--ff-only` stops instead of creating a merge commit you did not review.
7. B - a bare repository holds objects and references but no checked-out files.
8. C - `160000` is the gitlink mode that records a commit of another repository.

Chapter 9 - Inside Git: objects, references and configuration

9.1 Summary

- Everything Git stores is an object addressed by the SHA-1 hash of its content.
- There are four object types: blob, tree, commit and tag.
- Plumbing commands such as `hash-object`, `cat-file` and `ls-tree` expose that database directly.
- References are small files that point at objects, and branches, tags and HEAD are all references.
- `git gc` packs loose objects into packfiles, and `git fsck` reports unreachable ones.
- `git reflog` and `git fsck --lost-found` are the two tools that recover "deleted" commits.
- Configuration lives in three files, read from the most general to the most specific.
- Aliases are configuration entries that run a command with fixed options, and hooks are scripts that Git runs at defined moments.

9.2 Concepts

The four object types

Type	What it holds	Read with
Blob	the content of one file	`git cat-file -p <id>`
Tree	a directory listing: names, modes and identifiers	`git ls-tree <tree-ish>`
Commit	a tree, parents, author, committer and message	`git cat-file -p <commit>`
Tag	a tag object with tagger, date and message	`git cat-file -p <tag>`

Porcelain commands are the friendly ones this volume used; plumbing commands such as `hash-object`, `cat-file` and `ls-tree` work on the database itself. A commit points at one tree, the tree points at blobs and other trees, and every line is an identifier, so identical content is always the same object and `git hash-object` on the same content always prints the same identifier.

References are the only names

`.git/refs/heads/main` holds one line: a commit identifier. Moving a branch means rewriting that line; merging means writing a new commit and moving the line. `HEAD` is a symbolic reference: it contains the name of the branch you are on, not a commit. `git show-ref` lists the references, and `git symbolic-ref HEAD` prints what HEAD points at.

output

```
HEAD  ->  refs/heads/main  ->  9f2a86d...
tag v1.1.0 (annotated)     ->  tag object -> commit
refs/remotes/origin/main   ->  commit
```

Objects are first written as loose files, one per object. `git gc` compacts them into packfiles, where only the differences between similar objects are stored. `git fsck` walks every object and reports problems; `dangling` entries are not errors, they are objects that no reference points at any more, which is exactly what a reset, an amend or a dropped stash leaves behind. `git fsck --lost-found` writes them into `.git/lost-found` so they can be read again.

Configuration, aliases and hooks

Level	Command	File
System	`git config --system`	the installation's `gitconfig`
Global	`git config --global`	your account's `.gitconfig`
Local	`git config --local` (the default in a repository)	`.git/config`

The most specific value wins, so a repository can override your personal identity. An alias is any configuration entry under `alias.`, and it can contain a whole command with options. Hooks are executable files in `.git/hooks`, named after the moment they run: `pre-commit` before a commit is created, `commit-msg` after the message is written, and `pre-push` before objects are sent to a remote. Git installs only samples there, so a hook exists when you write the file and make it executable. A `pre-commit` hook that runs a formatter or a linter enforces the project's rules for every clone, though hooks are local, so teams that need enforcement on the server use a CI job instead.

9.3 Examples

Example 1: what a commit is made of.

shell

```
git rev-parse HEAD
git cat-file -t HEAD
git cat-file -p HEAD
```

output

```
9f2a86daf9661797730185ff2fc96fbd8fe35463
```

output

```
commit
```

output

```
tree c7369cc38297b1d225493ceadc8bd69db24b0525
parent aa4a02fcc6c1412037bd58713b8b010e8397be5b
author Ada Learner <ada@example.com> 1791669506 +0200
committer Ada Learner <ada@example.com> 1791669506 +0200

conf: add clash from main
```

One tree, one parent, two timestamps and the message. `git cat-file -t` names the type of any object, which is how you tell a tag object from a commit.

Example 2: a tree, and the identifier of content.

shell

```
git ls-tree HEAD
git hash-object README.md
git show HEAD:README.md | git hash-object --stdin
```

output

```
100644 blob 419d77ad8473cc5930c50261aaa5d2dac33c15e2□README.md
040000 tree bd49cf6f5acd6011d277c7ef3c3bd4e6fc8b20fe□vendored
```

output

```
419d77ad8473cc5930c50261aaa5d2dac33c15e2
419d77ad8473cc5930c50261aaa5d2dac33c15e2
```

Mode `040000` is a subdirectory and mode `160000` is a submodule. Two different routes to the same content, one identifier: that property is what makes sharing, deduplication and integrity checks possible.

Example 3: references, tags and a description of the current commit.

shell

```
git show-ref
git symbolic-ref HEAD
git describe --tags
```

output

```
9f2a86daf9661797730185ff2fc96fbd8fe35463 refs/heads/main
aa4a02fcc6c1412037bd58713b8b010e8397be5b refs/heads/note-worktree
f5b8e9639559fc62efb873d3561e09a95798e816 refs/heads/origin/main
5e65b79cf3a97efe9cc87b8f35915987ee92b6ed refs/tags/v1.1.0
```

output

```
refs/heads/main
```

output

```
v1.1.0-20-g9f2a86d
```

The tag line is the identifier of the tag object, not the commit, because `v1.1.0` is annotated. `describe` counts twenty commits past the tag.

Example 4: packing, integrity and an alias.

shell

```
git count-objects -v
git gc
git count-objects -v
git fsck
git config alias.lg "log --oneline --graph --decorate --all"
git lg
```

output

```
count: 29
size: 7
in-pack: 78
packs: 2
size-pack: 13
```

output

```
count: 0
size: 0
in-pack: 107
packs: 2
size-pack: 17
```

output

```
dangling tree 23f923bdd27c0fc26271fa951874daee2c326dde
dangling commit d578a963a6b755dcf52b55e627a947c49786448c
```

output

```
* 9f2a86d (HEAD -> main) conf: add clash from main
*   aa4a02f (note-worktree) Merge branch topic-note into the docs line
|\
| * 23aba28 docs: add an extra note
* | cbd2818 docs: describe the repository layout
```

The loose objects moved into the packfiles, and `fsck` reports the objects that no reference points at any more. The alias `lg` now prints the decorated graph, which is the view most

developers keep as their default log.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Editing files under `.git` by hand	references become inconsistent	use the plumbing commands, which keep the index and refs in step
Assuming `git gc` must be run	it is automatic and running it constantly wastes CPU	let `gc --auto` decide
Treating `dangling` in `git fsck` as corruption	it is normal after a reset or an amend	read the identifiers, then decide whether to recover them
Setting `user.email` locally and forgetting it	later commits in other projects use the wrong address	check `git config user.email` when starting a new project
Writing hooks that slow every commit	developers disable them	keep hooks fast, or run the slow checks in CI

9.5 Practice exercises

1. Name the four object types and the command that tells you which one an identifier points at.
2. What does `.git/refs/heads/main` contain, and what happens to it when you commit?
3. Why do two files with the same content have the same blob identifier?
4. Where do the three levels of configuration live, and which one wins?

Solutions

1. Blob, tree, commit, tag; `git cat-file -t <identifier>`.
2. One line: the identifier of the commit the branch points at. A commit rewrites that line with the new commit's identifier.
3. Because the identifier is the hash of the content alone, independent of name, history or location.
4. `--system` in the installation, `--global` in your account, `--local` in the repository; the most specific one wins, so local beats global beats system.

9.6 Mini test

1. Which object type stores the content of a file? - A) tree - B) blob - C) commit - D) tag
2. Which command prints the type of an object? - A) `git cat-file -t <id>` - B) `git show --type <id>` - C) `git hash-object -t <id>` - D) `git rev-parse --type <id>`
3. What does `git ls-tree HEAD` list? - A) the commits of the branch - B) the files of the tree that HEAD points at - C) the branches of the repository - D) the objects in the pack

4. Which command prints the branch HEAD points to? - A) `git branch --current` - B) `git symbolic-ref HEAD` - C) `git show HEAD` - D) `git rev-parse --head`
5. True or false: after `git gc`, objects are stored in packfiles instead of loose files.
6. What does `git fsck` list as `dangling`? - A) corrupt objects - B) objects that no reference points at any more - C) objects that are not packed - D) objects belonging to another repository
7. Which configuration level overrides the other two? - A) `--system` - B) `--global` - C) `--local` - D) none, the last one written wins
8. Where do client-side hooks live? - A) `.git/refs/hooks` - B) `.git/hooks` - C) `.git/config` - D) `.githooks`

Answer key and explanations

1. B - a blob is file content with no name; the tree supplies the name.
2. A - `-t` prints the type, `-p` prints the content.
3. B - the tree maps names to blob identifiers for one directory.
4. B - `git branch --current` also works; `symbolic-ref` is the plumbing form.
5. True - loose objects are replaced by packfiles after a `gc`.
6. B - a dangling object is usually what a reset or an amend left behind.
7. C - the local repository's configuration is the most specific one.
8. B - hooks are executable scripts named after the moment they run.

Cheat sheet

10.1 Setup and configuration

Command	Purpose
`git --version`	print the installed version
`git config --global user.name "Name"`	set the author name for every project
`git config --global user.email "mail"`	set the author email for every project
`git config --local --list`	read this repository's settings
`git config --global alias.lg "log --oneline --graph"`	create an alias
`git help <command>`	open the manual

10.2 Creating, recording and reading

Command	Purpose
`git init -b main`	start a repository with the branch `main`
`git clone <url> [dir]`	copy a remote repository
`git status` / `git status -sb`	the state of the working tree and the index
`git add <path>` / `git add -p`	stage a file, directory or selected hunks
`git commit -m "msg"`	record the index
`git commit -a -m "msg"`	stage tracked edits and commit
`git commit --amend`	replace the last commit
`git rm <path>`	delete a file and stage the deletion
`git log --oneline --graph --decorate --all`	the shape of the whole project
`git log -n 3 -p`	the last three commits with their patches
`git log --oneline -- <path>`	the history of one file
`git log main..feature`	what `feature` adds over `main`
`git show <rev>:<path>`	a file as it was at a revision
`git diff` / `git diff --staged` / `git diff HEAD`	working tree, index and commit
`git blame <file>`	the commit that last touched each line
`git grep -n "text"`	search the tracked content

10.3 Branches, merges, conflicts and tags

Command	Purpose
`git branch` / `git branch -v`	list branches, with their tips
`git switch -c <name>`	create and check out a branch
`git switch <name>`	check out an existing branch
`git merge <branch>`	merge into the current branch
`git merge --no-ff -m "msg" <branch>`	force a merge commit
`git merge --abort`	cancel a merge with conflicts
`git status`	list the unmerged paths during a conflict
`git checkout --ours -- <file>` / `--theirs`	take one side of a conflict
`git branch -d <name>`	delete a merged branch

10.4 Rebase and history

Command	Purpose
`git rebase <base>`	replay the branch onto `<base>`
`git rebase -i HEAD~3`	reorder, reword, squash or drop
`git rebase --continue` / `--abort` / `--skip`	drive an interrupted rebase
`git cherry-pick <rev>`	copy one commit

10.5 Remotes, hosting and sharing

Command	Purpose
`git remote -v`	list the remotes
`git remote show origin`	explain the tracking setup
`git fetch origin`	download and update `origin/*`
`git fetch --prune`	also delete stale remote-tracking branches
`git pull --ff-only`	fetch and fast-forward only
`git push -u origin <branch>`	publish a branch and record the upstream
`git push --force-with-lease`	rewrite safely after a rebase
`git submodule update --init --recursive`	populate submodules after a clone
`git bundle create all.bundle --all`	a portable file with the history

Final test

1. What are the four areas of a Git project?
2. Which command creates a repository whose first branch is called `main`? - A) `git init` - B) `git init -b main` - C) `git start main` - D) `git branch --init main`
3. What does `git add` do?
4. Which command prints the changes that the next commit would contain? - A) `git diff` - B) `git diff HEAD` - C) `git diff --staged` - D) `git status -s`
5. What does `??` mean in `git status --short` output?
6. Which command compares the working tree with the last commit? - A) `git diff --staged` - B) `git diff --cached` - C) `git diff HEAD` - D) `git diff --worktree`
7. True or false: `git commit -a` stages files that were never added.
8. What does `git log --oneline -- README.md` print?
9. Which command annotates a file with the commit that last changed each line? - A) `git annotate` - B) `git blame` - C) `git log -L` - D) `git who`
10. Which `.gitignore` pattern ignores every directory named `build` at any depth? - A) `/build` - B) `build` - C) `build/` - D) `**/build`

11. What does `git clean -ndX` print?
12. Which kind of tag is an object of its own with a message? - A) lightweight - B) annotated - C) signed only - D) remote
13. Which command discards an unstaged edit to a tracked file? - A) `git reset --hard` - B) `git restore <file>` - C) `git checkout -f` - D) `git clean <file>`
14. What does `git reset --mixed HEAD~1` leave behind? - A) the change staged in the index - B) the change as an unstaged edit in the working tree - C) nothing, the change is gone - D) a new commit
15. True or false: `git revert` rewrites the commit it undoes.
16. Which command finds where HEAD has been, in order to recover a commit? - A) `git log --all` - B) `git reflog` - C) `git fsck --full` - D) `git stash list`
17. What is a branch?
18. When does Git perform a fast-forward merge? - A) when both branches have new commits - B) when the target branch has no commits of its own since the branch was made - C) when `--no-ff` is given - D) never, `--ff` must be given
19. Which option forces a merge commit even when a fast-forward is possible? - A) `--ff-only` - B) `--squash` - C) `--no-ff` - D) `--octopus`
20. What do the lines between `<<<<<<< HEAD` and `=======` contain in a conflicted file?
21. Which command marks a conflicted file as resolved? - A) `git commit <file>` - B) `git add <file>` - C) `git merge <file>` - D) `git resolve <file>`
22. What does `git rebase main` do to the current branch's commits?
23. Which interactive rebase command combines a commit into the previous one and discards its message? - A) `squash` - B) `fixup` - C) `drop` - D) `reword`
24. Which command downloads remote changes without changing your branch? - A) `git pull` - B) `git fetch` - C) `git push` - D) `git sync`
25. What does `## main...origin/main [ahead 1]` mean?

11.1 Final test answers

1. The working tree, the index, the local repository and the remote repository.
2. B - `git init -b main` names the initial branch in the same command.
3. It writes the current content of the named paths into the index.
4. C - `git diff --staged` compares the index with the last commit.
5. The path is untracked: on disk, but not in the index and not in any commit.
6. C - `git diff HEAD` compares the working tree with the current commit.
7. False - `-a` stages tracked modifications only; a new file needs `git add`.
8. The commits that changed `README.md`, one line each.
9. B - `git blame` prefixes each line with the commit that produced it.
10. C - `build/` matches any depth; `/build` matches only the root.
11. The ignored files that would be removed; `-n` is the dry run.

12. B - an annotated tag is an object with tagger, date and message.
13. B - `git restore <file>` copies the indexed version over the file.
14. B - `--mixed` moves the branch and resets the index, keeping the edit.
15. False - revert records a new commit and leaves the original in place.
16. B - the reflog lists every move of HEAD with a reason.
17. A movable pointer to a commit, stored in `.git/refs/heads/<name>`.
18. B - the pointer can move forward, so no merge commit is needed.
19. C - `--no-ff` always records a merge commit.
20. The version on the current branch, `HEAD`.
21. B - staging the file records the resolution.
22. It replays them on top of `main`, giving each a new parent and identifier.
23. B - `fixup` keeps only the previous message.
24. B - `fetch` only adds objects and updates `origin/*`.
25. `main` has one commit the remote lacks, so `git push` would publish it.

Sources

12.1 The Pro Git book (git-scm.com/book)

- Landing page: https://git-scm.com/book
- About version control: https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control
- Recording changes: https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository
- Viewing the commit history: https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History
- Undoing things: https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things
- Basic branching and merging: https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

12.2 Manual pages used to check syntax and output

- git-log: https://git-scm.com/docs/git-log
- git-merge: https://git-scm.com/docs/git-merge
- gitignore: https://git-scm.com/docs/gitignore

12.3 Related pages